Software architecture document for project <SQL-Lite like DB>

Authors:

1. Sidorenko Sofia, 24215
2. Munkuev Vladislav, 24215
3. Obraztsov Dmitry, 24214
4. Romanenko Nikita, 24214

1. Goals and limitations

1.1. Key functional requirements

Use-Case 5.1.1: Changing the DB structure

Actors: CLI User

Goals: Changing DB structural components.

Preconditions: The system environment is installed and running correctly.

Trigger condition: The actor enters a valid DDL query (CREATE, DROP, etc.)

Main success scenario:

1. The actor inputs a valid query.
2. The system parses the query and returns a query object.
3. The query object is passed to the core driver.
4. The core driver creates the internal representation of the query.
5. The driver passes control to the file manager.
6. The file manager updates the required file structure.
7. On success, control is returned to the parser, which waits for the next query.

Alternative scenario A — Invalid query:

Trigger: The actor enters an invalid query.

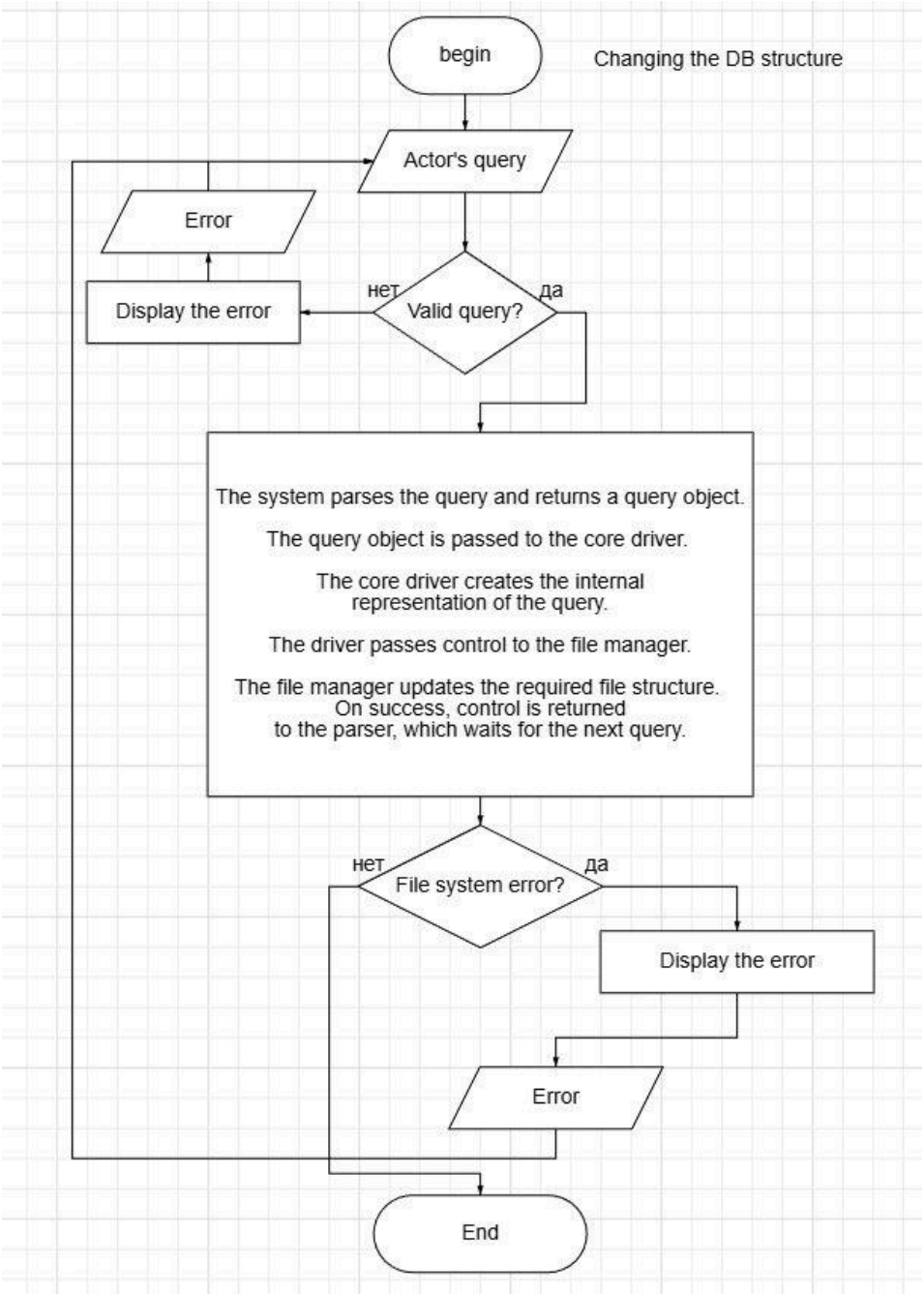
1. The parser detects a syntax or semantic error.

2. The parser displays an error message.
3. The system continues working normally.

Alternative scenario B — File system error:

Trigger: A failure occurs during file system operations.

1. The error is displayed.
2. The system continues working normally.



Use-Case 5.1.2: Modifying the DB data

Actors: CLI User

Goals: Modify the DB data using valid queries.

Preconditions: The database exists and its file representation is valid.

Trigger condition: The actor enters a valid DML or DDL query
(INSERT, UPDATE, etc.)

Main success scenario:

1. The actor inputs a valid query.
2. The system parses the query and returns a query object.
3. The query object is passed to the core driver.
4. The core driver builds an internal representation of the query.
5. Control is passed to the file manager.
6. The file manager updates the data accordingly.
7. On success, control is returned to the parser to await the next query.

Alternative scenario A — Invalid query:

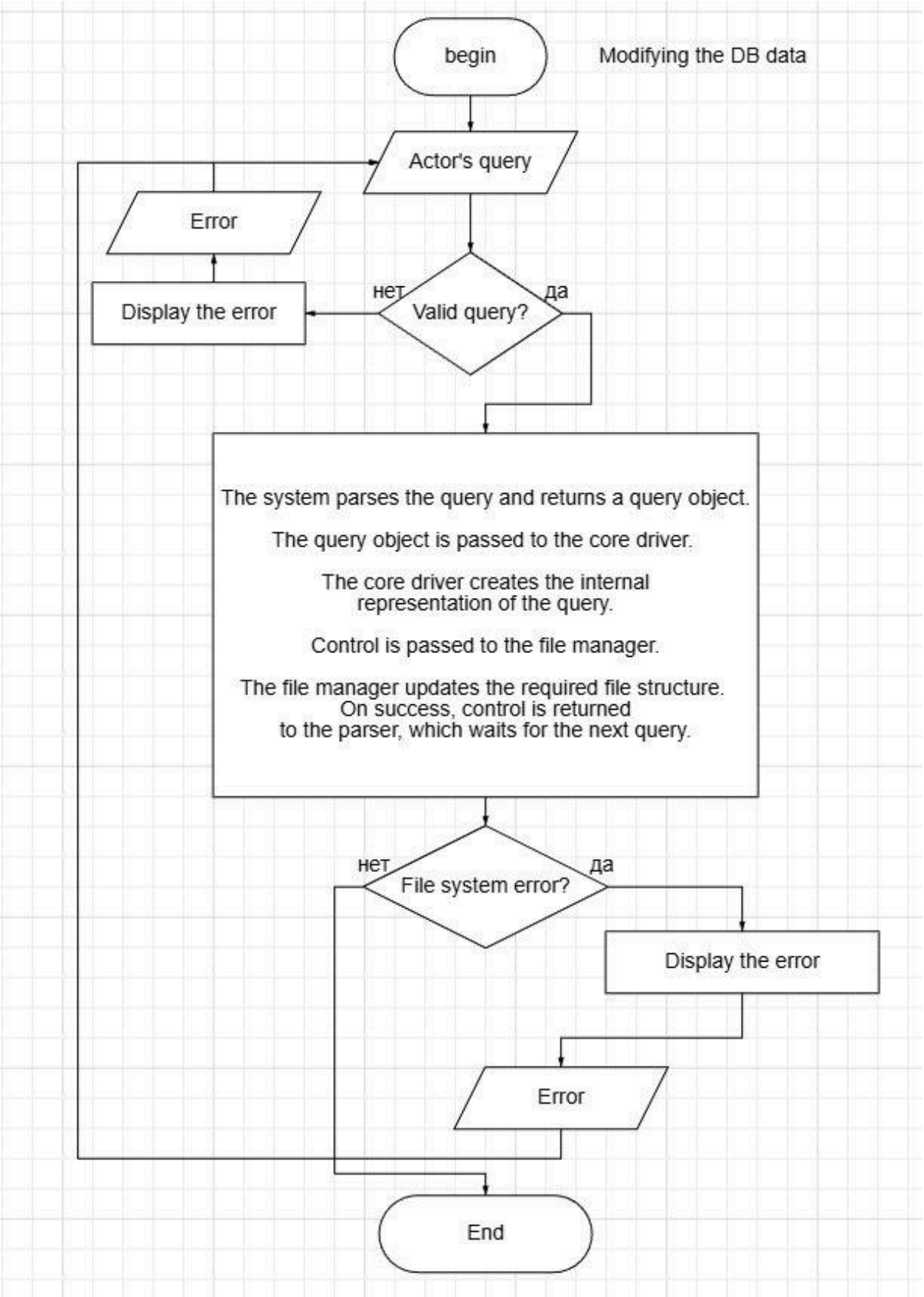
Trigger: Invalid query entered.

1. The parser detects an error and displays it.
2. The system continues normal operation.

Alternative scenario B — File system error:

Trigger: File system operation fails.

1. The system restores the previous data changes using the transaction log.
2. The error message is shown.
3. The system continues normal operation.


begin
Modifying the DB data
Actor's query
Error
Display the error
нет
Valid query?
да
The system parses the query and returns a query object.
The query object is passed to the core driver.
The core driver creates the internal representation of the query.
Control is passed to the file manager.
The file manager updates the required file structure. On success, control is returned to the parser, which waits for the next query.
нет
File system error?
да
Display the error
Error
End

Use-Case 5.1.3: Transaction control.

Actors: CLI User

Goals: Manage transactions and ensure data consistency.

Preconditions: The transaction log and data files are available and consistent.

Trigger condition: The actor enters valid TCL queries (BEGIN TRANSACTION, COMMIT, etc.)

Main success scenario:

1. The actor inputs a valid query.
2. The system parses the query and returns a query object.
3. The query object is passed to the core driver.
4. The driver creates an internal representation of the query.
5. Control is passed to the file manager.
6. The file manager commits or restores data changes using the transaction log.
7. On success, control is returned to the parser.
8. The system waits for the next query.

Alternative scenario A — Invalid query:

Trigger: Invalid query entered.

1. The parser detects an error and reports it.
2. The system continues normal operation.

Alternative scenario B — File system error:
Trigger: File system error.

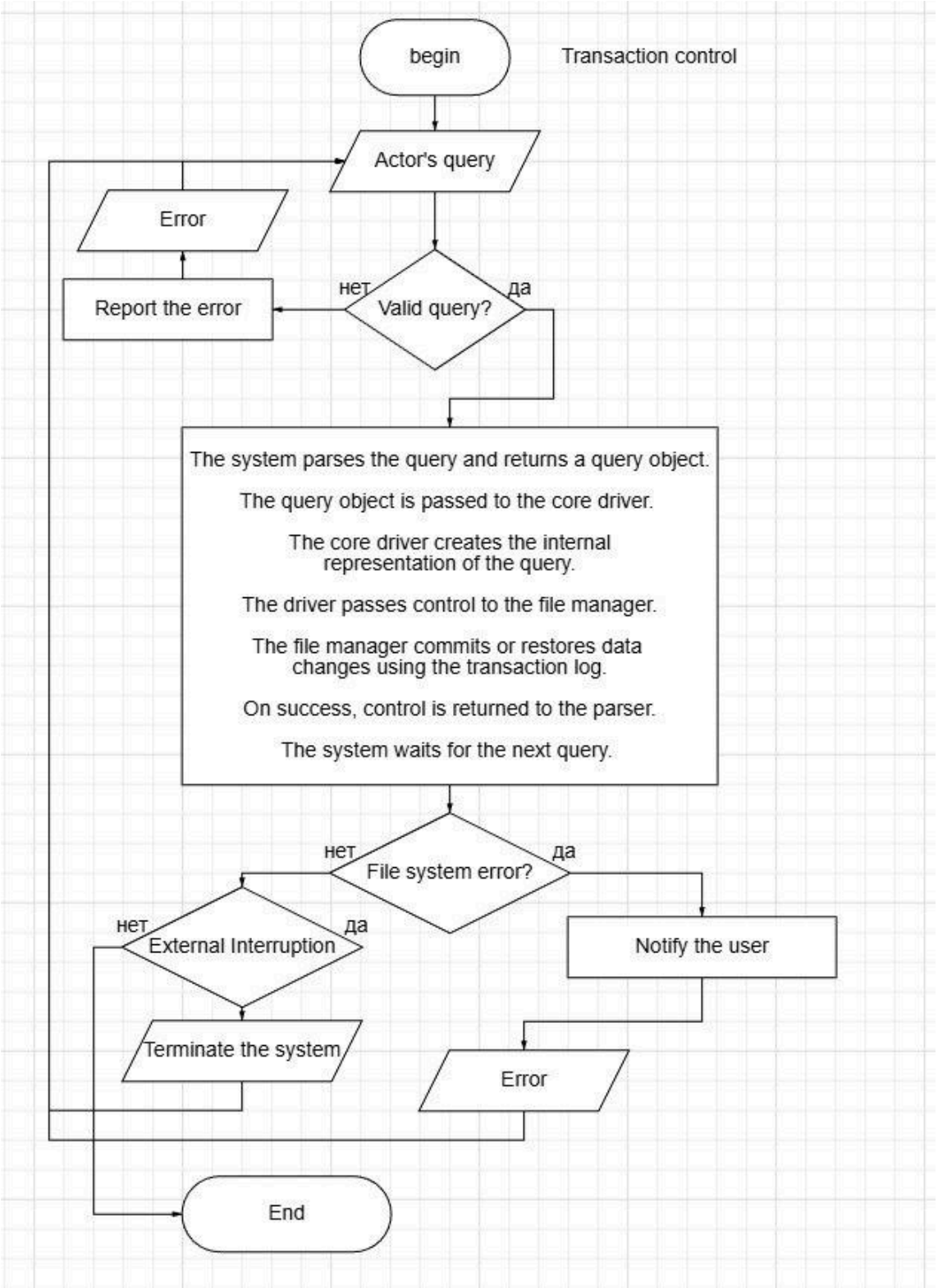
1. The system notifies the user of transaction failure.
2. The system continues operating.

Alternative scenario C — External

interruption: Trigger: External interruption

occurs.

1. The system terminates critically.



1.2. Non-functional requirements

Environment
Our library should work correctly on the following operating systems and their versions:
- Linux Ubuntu (ver. 24.04+),
- MacOS (ver. 13+)
- Windows (ver. 10+)

Implementation language: Java (ver. 17+)

Reliability
Recovery is implemented using simple logging. We want to log every request except SELECTs.

1.3. Architectural goals

- The DB must operate on Windows, Linux and MacOS
- The DB must be able to recover after crashing
- In our DB data stores in columns
- There must be JDBC support for using our system in Java programs
- Our system must successfully work with large size of database

1.4. Additional goals, restrictions and preferences

- Potential expansion to network mode.
- Optimized data storage in columns.